

1. Overview

PDFSolid Conversion SDK is a high-performance library designed for extracting and transforming the data within your PDF files, such as text, images, tables, links, and annotations, into various file formats. The Conversion SDK retains the original document layout and file data properties, helping you build reliable document conversion workflows.

Effortlessly integrate the PDFSolid Conversion SDK into your Node.js projects in just a few steps, and enable the following file format conversions:

- Convert PDF to Word (.docx)
- Convert PDF to Excel (.xlsx)
- Convert PDF to PowerPoint (.pptx)
- Convert PDF to HTML (.html)
- Convert PDF to CSV (.csv)
- Convert PDF to Image (.png, .jpg, .jpeg, .jpeg2000, .bmp, .tiff, .tga, .gif, .webp)
- Convert PDF to Plain Text (.txt)
- Convert PDF to Rich Text Format (.rtf)
- Convert PDF to Searchable PDF (.pdf)
- Convert PDF to OFD (.ofd)
- Convert PDF to Structured Data (.json)
- Convert PDF to Markdown (.md)

To enhance format conversion results, PDFSolid provides AI-powered document tools with the following capabilities:

- Optical Character Recognition (OCR)
- Layout Analysis
- Table Recognition

1.1 Why PDFSolid Conversion SDK

- Mature Technology

With years of technology accumulation, PDFSolid has established a complete product iteration mechanism to continuously improve conversion quality and reliability.

- Complete PDF and Format Conversion Functionalities

The SDK provides comprehensive conversion capabilities for common office, web, image, text, and structured data formats.

- High-quality Service

Professional service and technical support help developers respond quickly to integration questions and production issues.

- Independent Intellectual Property Rights

The technology is independent and compliant, helping enterprises conduct international business with lower copyright risk.

1.2 PDFSolid Conversion SDK for Node.js

The PDFSolid Conversion Node.js SDK wraps the native PDFSolid Conversion C/C++ runtime through a thin Node-API native addon instead of runtime FFI packages, so JavaScript code can call the native C API through a stable compiled addon.

The Node.js call chain is:

```
JavaScript
-> lib/index.js
-> Node-API addon: pdfsolid_conversion_node.node
-> libpdfsolidconversionsdk.so / pdfsolidconversionsdk.dll
-> PDFSolid Conversion C++ SDK
```

The current Node.js SDK package supports Linux x64.

1.3 License & Trial

PDFSolid Conversion SDK is a commercial SDK that requires a license to grant developers the right to develop and distribute their applications. In development mode, each license is only valid for one device ID. PDFSolid provides flexible licensing models. Please contact [our marketing team](#) for more information.

If you do not have a license, please contact the [PDFSolid Team](#) to obtain a trial license for PDFSolid Conversion SDK.

2. Get Started

2.1 Requirements

Before starting, make sure that you have already met the following prerequisites.

2.1.1 Get PDFSolid License Key

PDFSolid provides two types of license key: 30-day free trial license and commercial license.

How to Get Free Trial License

[Contact our sales team](#) and we will send you a 30-day free trial license for PDFSolid Conversion SDK.

How to Get Commercial License

PDFSolid Conversion SDK is a commercial SDK that requires a license for application release. Any documents, sample code, or source code distribution from the released package of PDFSolid to any third party is prohibited.

To get a commercial license for PDFSolid Conversion SDK, feel free to [contact our sales team](#).

For Node.js Conversion SDK, commercial licenses are verified through the SDK license verification API. In development mode, a license is bound to the developer device ID.

2.1.2 Download Conversion SDK

[Contact us](#) to obtain the PDFSolid Conversion SDK for Node.js.

2.1.3 System Requirements

Development Platform	System Requirements	Development Environment	Notice
Linux	Linux x64	Node.js 16 or later	Samples have been tested on Ubuntu 20.04.

The Node.js SDK does not require `ffi-napi` or other runtime FFI packages. It uses a Node-API addon packaged with the release bundle.

The release package already contains `lib/`, `resource/models/`, sample files, API reference, developer guide, legal information, and release notes. No `npm install` step is required when running the bundled demo from the release package.

2.2 SDK Package Structure

The PDFSolid Conversion Node.js release package contains the following files:

```
lib/
  index.js
  index.d.ts
  pdfsolid_conversion_node.node
  libpdfsolidconversionsdk.so
  libDocumentAI.so.4.0.0
  libonnxruntime.so.1.18.0
  libopencv_world.so.410
resource/
  models/
    documentai.model
samples/
  demo/
    demo.js
  license.xml
  input_files/
    word.pdf
    excel.pdf
    powerpoint.pdf
  output_files/          # sample output directory
doc/
  api_reference_node.html
  developer_guide_node.md
  html/
legal.txt
release_notes.txt
```

The main directories are:

- `lib/index.js`: SDK entry point loaded directly by the release sample. It loads the native addon from the same directory.

- `lib/index.d.ts`: TypeScript declarations for conversion options, callbacks, and exported functions.
- `lib/pdfsolid_conversion_node.node`: compiled Node-API addon.
- `lib/*.so`: native runtime libraries for Linux.
- `resource/models/`: packaged DocumentAI model used by the demo.
- `samples/demo/demo.js`: release sample entry point.
- `samples/input_files/`: bundled sample PDFs.
- `samples/output_files/`: sample output directory provided by the release package.
- `samples/license.xml`: sample license file used by default.
- `doc/`: API reference and developer guide.
- `legal.txt`: legal and copyright information.
- `release_notes.txt`: release information.

Load the Node.js SDK

The release package already contains the SDK runtime under `lib/`, so no installation step is required for the bundled samples.

When running samples from the release package, run from the package root directory:

```
cd /path/to/pdfsolid_conversion_node
node samples/demo/demo.js
```

The sample loads the SDK with:

```
const sdk = require("../lib/index.js");
```

2.3 Apply the License Key

If you do not have a license key, please check out [how to obtain a license key](#).

PDFSolid Conversion SDK currently supports offline authentication to verify license keys.

2.3.1 Copy the License Key

Accurately obtaining the license key is crucial for applying the license.

1. In the email you received, locate the XML file containing the license key.
2. Open the XML file, and determine the license type based on the `<type>` field. If `<type>online</type>` is present, it indicates an online license. If `<type>offline</type>` is present or if the field is absent, it indicates an offline license.
3. Copy the value located at the `LICENSE_KEY` position within the `<key>LICENSE_KEY</key>` field. This is your license key.

2.3.2 Apply the License Key

Use `sdk.licenseverify()` before initializing the SDK and starting conversions.

```
const sdk = require("../lib/index.js");

const license = "LICENSE_KEY";
const deviceId = "DEVICE_ID";
const appId = "com.example.application";

const code = sdk.licenseVerify(license, deviceId, appId);
if (code !== 0) {
  throw new Error(`license verification failed: ${code}`);
}

sdk.initialize();
```

You can also read license information from environment variables in command-line tools:

```
const license = process.env.PDFSOLID_LICENSE;
const deviceId = process.env.PDFSOLID_DEVICE_ID || "";
const appId = process.env.PDFSOLID_APP_ID || "com.example.application";
```

The sample scripts also accept `--license`, `--device-id`, and `--app-id`. When these options are not provided, they read the same environment variables and fall back to local demo values packaged in the samples.

2.4 How to Run a Demo

2.4.1 Quick Verification

The release package provides `samples/demo/demo.js` as the sample entry point. With no input PDFs, the sample converts the bundled PDFs under `samples/input_files` to Word, Excel, PowerPoint, CSV, HTML, RTF, image, TXT, JSON, Markdown, searchable PDF, and OFD outputs under `samples/output_files`.

```
cd /path/to/pdfsolid_conversion_node
node samples/demo/demo.js
```

Expected key output:

```
licenseVerify => 0
version => 1.2.0
setDocumentAIModel => 0
pdf to word: result=0, output=...
all conversion tasks finished
```

2.4.2 Multi-file Demo

The sample also supports multi-file conversion. Each worker process initializes the SDK independently.

```
cd /path/to/pdfsolid_conversion_node
node samples/demo/demo.js \
  --threads 2 \
  samples/input_files/word.pdf \
  samples/input_files/excel.pdf
```

By default, the sample writes conversion results to `samples/output_files`. Use `--output` only when you want to write results to another directory.

The sample uses `samples/license.xml` by default. Override it with `--license` or `PDFSOLID_LICENSE` when testing another license XML path or license key.

```
node samples/demo/demo.js \
  --license /path/to/license.xml \
  samples/input_files/word.pdf
```

The sample loads the packaged DocumentAI model by default:

```
resource/models/documentai.model
```

Override it with `--model` or `PDFSOLID_MODEL_PATH`:

```
node samples/demo/demo.js \
  --model /path/to/documentai.model \
  --threads 2 \
  samples/input_files/word.pdf
```

3. Conversion Guides

PDFSolid Conversion SDK allows developers to use a simple API to convert PDF to common file formats like Word, Excel, PPT, HTML, CSV, PNG, JPEG, RTF, TXT, Searchable PDF, OFD, JSON, and Markdown. It provides a wide range of customized conversion options, such as whether to include images or annotations in PDF documents, whether to enable OCR, whether to enable layout analysis, and more.

All Node.js conversion APIs follow the same basic signature:

```
const result = sdk.startPDFToWord(
  inputFilePath,
  password,
  outputFilePath,
  options,
  callback
);
```

`password` can be an empty string when the input PDF is not encrypted. Each conversion method returns an SDK error code. `0` means the conversion succeeded.

3.1 Initialize Library Resources

Overview

Initialize the necessary file and memory resources required by the PDFSolid Conversion SDK.

The Node.js SDK initializes with its packaged internal resource directory when `initialize()` is called without arguments. You can still pass a custom resource path when needed.

Notes

- You must verify the license and initialize SDK resources before calling any conversion interface.
- When using OCR, Layout Analysis, Table Recognition, PDF to Searchable PDF, or PDF to OFD, make sure the DocumentAI model is available and loaded when the feature requires it.
- Call `release()` after conversions complete so native SDK resources are freed.

Example

```
sdk.initialize();

// Or use an external resource directory.
sdk.initialize("/path/to/resource");
```

3.2 Set DocumentAI Model

Overview

Before using OCR, Layout Analysis, or Table Recognition, you need to set the DocumentAI model path first.

`setDocumentAIModel` supports an optional `gpuId` parameter used to specify the GPU device index for the AI model. When `gpuId` is `-1`, GPU acceleration is disabled.

Set AI Model Instance Count

If you need to control the number of Layout Analysis and Table Recognition model instances, call `setDocumentAIModelCount`.

Sample

```
const modelPath = "/path/to/documentai.model";
sdk.setDocumentAIModelCount(1, 1);
const code = sdk.setDocumentAIModel(modelPath, -1);
if (code !== 0) {
  throw new Error(`setDocumentAIModel failed: ${code}`);
}
```

When the conversion work is done, call `releaseDocumentAIModel()` if a DocumentAI model was loaded.

```
sdk.releaseDocumentAIModel();
```

3.3 Get Conversion Progress

PDFSolid Conversion SDK obtains conversion progress through callback functions. The following example demonstrates how to get conversion progress while performing a PDF to Word task.

```
const callback = {
  onProgress(currentPage, totalPages) {
    console.log(`progress: ${currentPage}/${totalPages}`);
  }
};
```

Pass the callback as the fifth conversion argument:

```
const result = sdk.startPDFToWord(
  inputFilePath,
  "",
  outputFilePath,
  {},
  callback
);
```

3.4 Cancel Conversion Task

PDFSolid Conversion SDK supports interrupting an ongoing conversion task. Return `true` from `isCancelled()` to cancel the current task, or `false` to continue.

```
let cancelRequested = false;

const callback = {
  isCancelled() {
    return cancelRequested;
  }
};
```

The current addon calls JavaScript callbacks synchronously from the conversion call thread. If the native SDK triggers callbacks from internal worker threads in the future, the addon must use `napi_threadsafe_function` instead of direct `napi_call_function()`.

3.5 Select Page Range for Conversion

PDFSolid Conversion SDK supports converting a specified page range. When an empty string is passed, all pages will be converted. If the page range exceeds one page, you can also choose to enable the `outputDocumentPerPage` option to output each PDF page as a separate file.

```
const options = {
  pageRanges: "1-3,5,7-9",
  outputDocumentPerPage: false
};
```

3.6 Conversion Options: Contain Image & Annotation

Overview

In the process of converting PDF documents into various formats, PDFSolid Conversion SDK offers options to determine whether images are included in the generated document and whether annotations from the PDF file are retained.

In the Node.js SDK, use `containImage` and `containAnnotation` on the conversion options object.

Sample

```
const options = {
  containImage: true,
  containAnnotation: true,
  enableAiLayout: true,
  enableocr: false
};

const result = sdk.startPDFToWord(inputFilePath, "", outputFilePath, options);
```

3.7 Page Layout Mode

In certain formats, the page layout mode plays a key role in the quality of the converted document. PDFSolid Conversion SDK supports two layout modes: Flow Layout and Box Layout.

- **Flow Layout:** Uses paragraph indentations, columns, and tab positions to adjust the content. This layout is suitable for editable documents.
- **Box Layout:** Places each element based on the original PDF coordinate system. This layout is suitable for documents requiring precise reproduction.

Page layout modes are commonly used for PDF to Word and PDF to HTML.

```
const PageLayoutMode = {
  BOX: 0,
  FLOW: 1
};

const options = {
  pageLayoutMode: PageLayoutMode.FLOW
};

const result = sdk.startPDFToWord(inputFilePath, "", outputFilePath, options);
```

3.8 OCR

Overview

OCR (Optical Character Recognition) is the process of converting images of typed, handwritten, or printed text into machine-encoded text.

OCR is commonly used for text recognition and extraction from scanned PDF files, photographs of documents, scene photos, invoices, receipts, and other image-based documents.

The following features support OCR:

- PDF to Word
- PDF to Excel
- PDF to PowerPoint
- PDF to HTML
- PDF to RTF
- PDF to TXT
- PDF to Searchable PDF
- PDF to OFD
- Extract PDF to JSON
- Extract PDF to Markdown

Set OCR Language

Use `languages` to specify OCR languages. The value is an array of numeric OCR language constants.

```
const OCRLanguage = {
  CHINESE: 1,
  ENGLISH: 3,
  AUTO: 16
};

const options = {
  enableocr: true,
  languages: [OCRLanguage.ENGLISH, OCRLanguage.CHINESE]
};
```

OCR Options

Use `ocrOption` to control OCR processing scope.

```
const OCROption = {
  INVALID_CHARACTER: 0,
  SCAN_PAGE: 1,
  INVALID_CHARACTER_AND_SCAN_PAGE: 2,
  ALL: 3
};

options.ocrOption = OCROption.ALL;
```

Preserve Page Background

When OCR is enabled, use `containPageBackgroundImage` to control whether page background images are preserved.

```
options.containPageBackgroundImage = true;
```

Sample

```
sdk.setDocumentAIModel("/path/to/documentai.model", -1);

const options = {
  enableOcr: true,
  languages: [3],
  ocrOption: 3
};

const result = sdk.startPDFToWord(inputFilePath, "", outputFilePath, options);
```

3.9 Layout Analysis

Overview

Layout Analysis detects the layout structure of a PDF page and helps improve output quality for complex pages.

Use `enableAiLayout` to enable or disable AI layout analysis.

Sample

```
sdk.setDocumentAIModel("/path/to/documentai.model", -1);

const options = {
  enableAiLayout: true
};

const result = sdk.startPDFToWord(inputFilePath, "", outputFilePath, options);
```

3.10 Table Recognition

Overview

Table Recognition detects table structures and helps preserve table layout in output documents and extracted data.

Use `enableAiTableRecognition` to enable table recognition.

Sample

```
sdk.setDocumentAIModel("/path/to/documentai.model", -1);

const options = {
  enableAiLayout: true,
  enableAiTableRecognition: true
};

const result = sdk.startPDFToExcel(inputFilePath, "", outputFilePath, options);
```

3.11 Use Custom AI Models via Callbacks

Overview

The native C/C++ SDK exposes callback hooks that allow OCR, Layout Analysis, and Table Recognition results to be supplied by a custom AI engine. The current Node.js addon exposes progress and cancellation callbacks, but does not expose these custom AI callback extension points yet.

For Node.js integrations, use the packaged DocumentAI model through `setDocumentAIModel()` when OCR, Layout Analysis, Table Recognition, PDF to Searchable PDF, PDF to OFD, JSON extraction, or Markdown extraction requires AI capabilities.

Notice

- Custom AI callback hooks are available in the native C/C++ SDK v1.2.0 or later.
- The Node.js SDK keeps the native callback surface intentionally small and currently supports `onProgress` and `isCancelled` only.
- If custom AI callbacks are added to the Node.js addon later, they must be implemented with Node-API thread-safety in mind.

3.12 Output Font Option

Overview

Some output formats support specifying a preferred output font. Use `fontName` in the conversion options object.

Supported Formats

- PDF to Word
- PDF to Excel
- PDF to PowerPoint
- PDF to HTML
- PDF to RTF

Example

```
const options = {  
  fontName: "Arial"  
};
```

3.13 Convert PDF to Word

Overview

Converting PDF to Word is an operation that converts a PDF file into an editable Word file. By converting PDF to Word, you can edit, modify, insert, or delete text and images, and adjust layout and properties.

Sample

```
const options = {
  formulaToImage: true,
  transparentText: true
};

const result = sdk.startPDFToWord(
  inputFilePath,
  "",
  outputFilePath,
  options
);
```

3.14 Convert PDF to Excel

Overview

PDFSolid Conversion SDK supports converting PDF documents to Microsoft Excel format (.xlsx). By extracting, parsing, and importing data from PDF into Excel, users can further edit, analyze, or share Excel files.

Set the Content Options for Excel

When converting PDF files to Excel files, the following options directly affect content written to the Excel file.

- `excelAllContent`: whether the converted XLSX file contains all content in the PDF.
- `excelCsvFormat`: whether to save table data in CSV format.
- `excelWorksheetOption`: worksheet layout option.

Option	Value	Description
<code>FOR_TABLE</code>	<code>0</code>	Create one sheet per table.
<code>FOR_PAGE</code>	<code>1</code>	Create one sheet per page.
<code>FOR_DOCUMENT</code>	<code>2</code>	Create one sheet for the entire document.

Sample

```
const options = {
  excelWorksheetOption: 0
};

const result = sdk.startPDFToExcel(
  inputFilePath,
  "",
  outputFilePath,
  options
);
```

3.15 Convert PDF to PowerPoint

Overview

PDFSolid Conversion SDK provides the function of converting PDF files to PowerPoint files and restoring the layout and format of the original document.

Sample

```
const result = sdk.startPDFToPpt(inputFilePath, "", outputFilePath, {});
```

3.16 Convert PDF to HTML

Overview

PDFSolid Conversion SDK provides the PDF to HTML function, which can convert PDF files to HTML files while maintaining the layout and format of the original document.

HTML Options

Option	Value	Description
<code>SINGLE_PAGE</code>	0	Convert the entire PDF into a single HTML file.
<code>SINGLE_PAGE_WITH_BOOKMARK</code>	1	Convert the PDF into a single HTML file with a bookmark navigation bar.
<code>MULTI_PAGE</code>	2	Convert the PDF into multiple HTML files, one file per page.
<code>MULTI_PAGE_WITH_BOOKMARK</code>	3	Convert the PDF into multiple HTML files with an outline page for navigation.

Sample

```
const options = {
  htmlOption: 0
};

const result = sdk.startPDFToHtml(
  inputFilePath,
  "",
  outputFilePath,
  options
);
```

3.17 Convert PDF to CSV

Overview

PDFSolid Conversion SDK supports converting PDF documents to CSV (Comma-Separated Values). In the Node.js SDK, CSV output is controlled by Excel conversion options.

Sample

```
const options = {
  excelCsvFormat: true,
  autoCreateFolder: true
};

const result = sdk.startPDFToExcel(
  inputFilePath,
  "",
  outputFilePath,
  options
);
```

3.18 Convert PDF to Image

Overview

PDFSolid Conversion SDK provides an API for converting PDFs to images.

Supported image formats include JPG, JPEG, JPEG2000, PNG, BMP, TIFF, TGA, GIF, and WEBP.

Sample

```
const ImageType = {
  JPG: 0,
  JPEG: 1,
  JPEG2000: 2,
  PNG: 3,
  BMP: 4,
  TIFF: 5,
  TGA: 6,
  GIF: 7,
  WEBP: 8
};

const options = {
  imageType: ImageType.PNG,
  imageColorMode: 0,
  imageScaling: 1.0
};

const result = sdk.startPDFToImage(
  inputFilePath,
  "",
  outputDirectory,
  options
);
```

3.19 Convert PDF to RTF

Overview

PDFSolid Conversion SDK supports converting PDF documents to Rich Text Format (.rtf).

Sample

```
const result = sdk.startPDFToRtf(inputFilePath, "", outputFilePath, {});
```

3.20 Convert PDF to TXT

Overview

PDFSolid Conversion SDK supports converting PDF documents to plain text files.

Use `txtTableFormat` to control whether table content is formatted in TXT output.

Sample

```
const options = {  
  txtTableFormat: true  
};  
  
const result = sdk.startPDFToTxt(inputFilePath, "", outputFilePath, options);
```

3.21 Convert PDF to Searchable PDF

Overview

PDFSolid Conversion SDK supports converting scanned or image-based PDF documents into searchable PDF files.

This feature usually requires OCR and DocumentAI model resources.

Sample

```
sdk.setDocumentAIModel("/path/to/documentai.model", -1);  
  
const options = {  
  enableOcr: true,  
  languages: [3]  
};  
  
const result = sdk.startPDFToSearchablePDF(  
  inputFilePath,  
  "",  
  outputFilePath,  
  options  
);
```

3.22 Convert PDF to OFD

Overview

PDFSolid Conversion SDK supports converting PDF documents to OFD format.

Sample

```
const result = sdk.startPDFToOfd(inputFilePath, "", outputFilePath, {});
```

3.23 Releasing Library Resources

Overview

Releases the files and memory resources occupied by the PDFSolid Conversion SDK.

Notice

- After calling `release()`, the SDK must be initialized again before another conversion task can run.
- When you only want to release resources occupied by the AI model, call `releaseDocumentAIModel()` instead of releasing all SDK resources.
- If a DocumentAI model was loaded, release it before or during SDK cleanup.

Sample

```
// Release AI model resources.  
sdk.releaseDocumentAIModel();  
  
// Release library resources.  
sdk.release();
```

4 Data Extraction Guide

Unleash the power of data with PDFSolid Conversion SDK's data extraction APIs to detect, recognize, analyze, and extract PDF text, images, tables, and document structure.

4.1 Extract PDF To JSON

Overview

PDFSolid Conversion SDK supports extracting PDF content into structured JSON data.

Use `jsonContainTable` to control whether table data is included in JSON output.

Sample

```
const options = {
  jsonContainTable: true,
  enableAiLayout: true,
  enableAiTableRecognition: true
};

const result = sdk.startPDFToJson(
  inputFilePath,
  "",
  outputFilePath,
  options
);
```

4.2 Extract PDF To Markdown

Overview

PDFSolid Conversion SDK supports extracting PDF content into Markdown format.

Sample

```
const options = {
  containImage: true,
  containAnnotation: true
};

const result = sdk.startPDFToMarkdown(
  inputFilePath,
  "",
  outputFilePath,
  options
);
```

5. Support

5.1 FAQ

- Does OCR work on x86 architecture?
Currently, OCR only works on x64 architecture.
- Do Node.js conversion APIs block the event loop?
Yes. The Node.js SDK uses synchronous conversion APIs. For server workloads, run conversions in worker threads or child processes to avoid blocking the main event loop.
- How should concurrent conversions be handled?
The release sample `samples/demo/demo.js` uses child processes so each conversion worker has its own SDK initialization and native runtime state.
- Does the Node.js SDK expose custom AI callback hooks?
The current addon supports progress and cancellation callbacks. The custom AI callback extension points available in the native C/C++ SDK are not exposed by the Node.js addon yet.
- When should SDK resources be released?

Always call `release()` after conversion work completes. If a DocumentAI model was loaded, call `releaseDocumentAIModel()` before or during cleanup.

5.2 Contact Us

Thanks for your interest in PDFSolid Conversion SDK, the easy-to-use and powerful development solution. If you encounter technical questions or bug issues when using PDFSolid Conversion SDK, please submit the problem report to the [PDFSolid team](#). More information as follows would help us to solve your problem:

- PDFSolid Conversion SDK product and version.
- Your operating system and Node.js version.
- Detailed descriptions of the problem.
- Any other related information, such as an error screenshot.

Contact Information

- Home link: <https://www.pdfsolid.com>
- Technical Support: <https://www.pdfsolid.com/support>
- Email: support@pdfsolid.com

Thanks,
The PDFSolid Team